

Apache Spark

An Introduction to Big Data Processing with PySpark

Contents

1. What Apache Spark is
2. Apache Spark EcoSystem
3. Spark Runtime Components
4. Introduction to PySpark
5. Key Advantages of PySpark
6. Spark Core Module
7. Spark Entry Points
8. Resilient Distributed Datasets
9. DataFrame & Dataset APIs
10. Directed Acyclic Graph
11. PySpark Data Handling

What Apache Spark is

- Apache Spark is described as a multi-language engine for executing data engineering, data science, and machine learning on single-node machines or clusters.
- It is characterized as Simple, Fast, Scalable, and Unified.
- A general-purpose cluster computing system well-suited for large-scale data processing.
- It performs significantly better than Hadoop MapReduce



Apache Spark EcoSystem



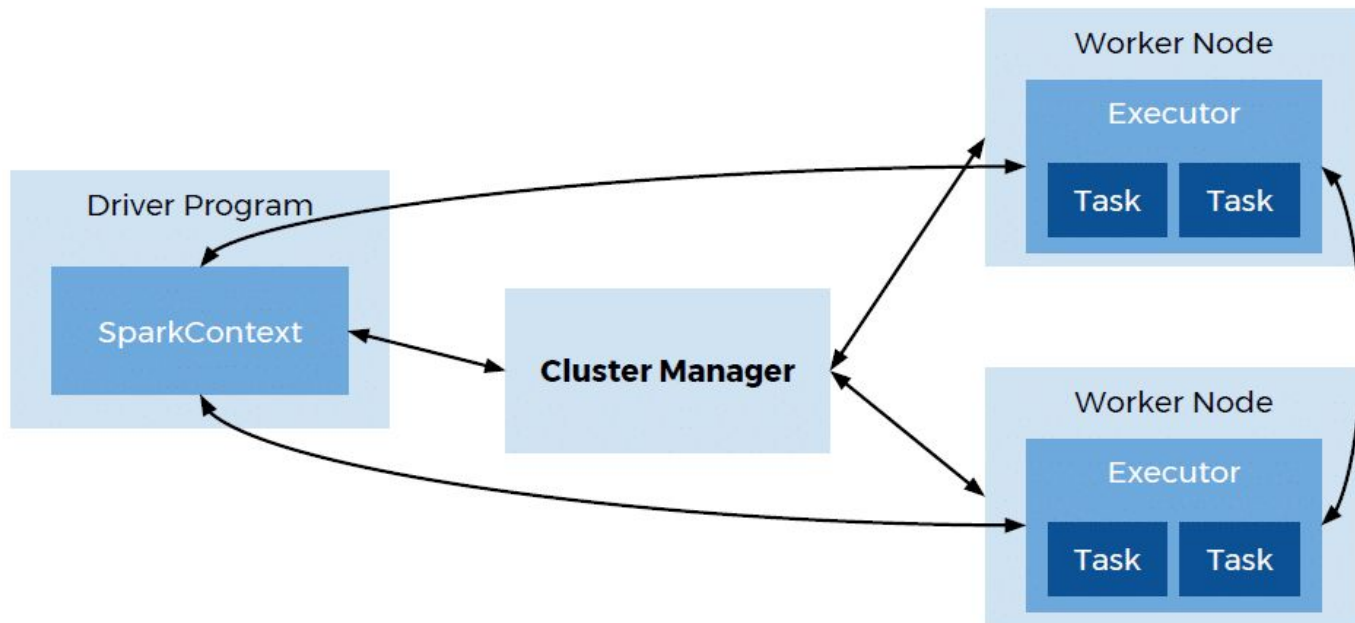
Apache Spark EcoSystem

- **Spark SQL:** execute fast, distributed ANSI SQL queries for dashboarding and ad-hoc reporting.
- **Spark Streaming:** extension of the core Spark API for processing real-time streaming data.
- **MLlib** (Machine Learning): for training machine learning algorithms.
- **GraphX** (Graph): processing data that follows graph data structures.
- **Pandas on Spark:** Allows users to leverage the pandas API while working with Spark DataFrames.
- **PySpark:** a Python Library built on top of Apache Spark.

Spark Runtime Components

- **Driver Program:** This program runs on the master system. It contains the main routine of the Spark application and performs actions/transformations. The driver program negotiates with the cluster manager for scheduling tasks.
- **Cluster Manager:** The cluster manager (e.g., Standalone, Yarn, Mesos, Kubernetes) allocates resources to tasks.
- **Master Node:** In the Spark cluster, the Master Node requests resources and makes them available to the Spark Driver.
- **Worker Node:** Worker nodes (also referred to as Slave Nodes) run the application code within the cluster.
- **Executor:** Executors run tasks on the worker nodes. After running tasks, they send results back to the driver.
- **Task:** A task represents a computation on a data partition within a stage of an RDD.

Spark Runtime Components



Introduction to PySpark

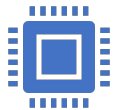
- PySpark is a Python library that utilizes Apache Spark.
Designed for parallel data processing across clusters.
Ideal for big data applications.



Key Advantages of PySpark



Distributed
computing



In-memory
computation



Extensive APIs



Python
integration

- Supports distributed computing for large datasets.
- Offers in-memory computation capabilities.
- Provides extensive APIs for various applications.
- Integrates well with Python libraries like Pandas and NumPy.

Spark Core Module

- At the heart of Spark is the Spark **Core module**.
- It serves as the engine that underpins the entire system and its various components.
- Responsible for:
 - Task scheduling
 - Memory management
 - Fault recovery
 - Interacting with storage systems

Spark Core Module

- Spark Core operates on Resilient Distributed Datasets (RDDs) Spark's fundamental data structure stored in memory by default which enables fault-tolerant, parallel processing through transformations and lineage-based recovery.
- Spark Core acts as the execution engine for other higher-level components and libraries built on top of it.
- Spark's architecture uses RDDs and a Directed Acyclic Graph (DAG) to define and schedule computations, with the DAG and Task schedulers coordinating parallel execution through stages and tasks based on transformation sequences and shuffle boundaries.
- Provides support for higher-level APIs and libraries

Spark Entry Points

- Spark Session and Spark Context are crucial components of PySpark.
- Enable distributed data processing.
- Provide high-level API to interact with data.
- **SparkContext**
 - Entry point for Spark functionality, connects to Spark cluster.
- **SparkSession**
 - Unified entry point for structured data processing.
- Use **SparkContext** for low-level RDD operations and **SparkSession** for high-level structured data tasks.

SparkContext

- Entry point to Spark functionality.
- Connects to a Spark cluster.
- Serves as the client-side driver program.
- Enables the applications to access Spark's distributed computation capabilities.

```
from pyspark import SparkContext  
sc = SparkContext(master="local", appName="pyspark-intro")
```

SparkSession

- Unified entry point for structured data processing.
- Includes the features of both spark context and SQL context.

```
from pyspark.sql import SparkSession  
spark = SparkSession.builder.appName("pyspark-intro").getOrCreate
```

RDDs: Resilient Distributed Datasets

- Immutable distributed collections of data elements that can be processed in parallel across a cluster.
- Form the core of Spark's distributed data processing capabilities.
- RDDs are fault tolerant using lineage information.
- RDDs do not have any pre-defined structure.

RDDs: Resilient Distributed Datasets

- RDDs can be created from external data sources like HDFS, RDBMS, or S3, or from existing in-memory data.

- To Create RDD From List:

```
data = [1, 2, 3, 4, 5]
```

```
rdd = sc.parallelize(data)
```

RDDs: Resilient Distributed Datasets

RDDs processing model contains two crucial forms: **Transformations** and **Actions**.

1. Transformations:

- Lazy operations that return new RDDs from existing ones.
- Lazy means that they are not executed immediately but their execution plan is recorded.
- Common RDD transformations include:
 - `map`, `filter`, `flat_map`, and `reduced_by_key`.

RDDs: Resilient Distributed Datasets

Example Transformations:

```
squared_rdd = rdd.map(lambda x: x ** 2)
```

```
even_rdd = rdd.filter(lambda x: x % 2 == 0)
```

RDDs: Resilient Distributed Datasets

2. Actions:

- Trigger execution and return results.
- Eagerly evaluated and imitate the computation on the RDD.
- Some common RDD actions include:
 - collect, count, reduce, and save_as_text_file.

RDDs: Resilient Distributed Datasets

Example Actions:

```
collected = squared_rdd.collect()
```

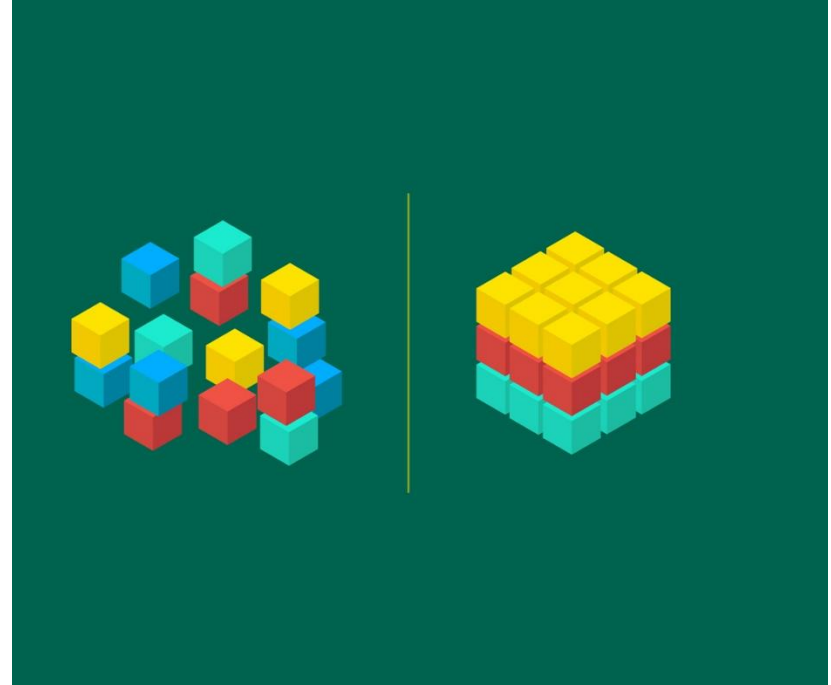
```
count = squared_rdd.count()
```

DataFrame & Dataset APIs

- Abstractions built on top of RDDs.
- Made working with distributed data accessible and easier.
- Use **Dataset API** for strongly-typed, structured schema.
DataFrame API allows loading structured data from sources like CSV.

Structured vs Semi-Structured Data

- **Structured** data is organized in a tabular format using DataFrames.
- **Semi-structured** data includes formats like JSON and XML and nested structures.



Dataset API

- Use structured schema for strongly-typed data.
- Combines the benefits of both DataFrame and RDD APIs.
- Provides a typesafe programming interface.

Dataset API

```
schema = StructType([  
    StructField("name", StringType(), True),  
    StructField("age", IntegerType(), True),  
    StructField("salary", IntegerType(), True)  
])
```

DataFrame API

- Data organized into named columns (tables).
- Provides a natural and efficient way to handle structured data compared to RDDs.
- Example:

```
dataframe = spark.read.csv("data.csv", header=True, inferSchema=True)
```

Execution Model

- Spark uses a sophisticated execution engine based on DAGs.
- **DAG (Directed Acyclic Graph):**
 - Spark creates a graph denoting the sequence of operations performed on data. The architecture itself is based on RDD and DAG abstractions. When an action is called on an RDD, Spark submits the graph to the DAG scheduler.
- **Transformations and Actions:**
 - Spark operations are classified into transformations and actions.
- **Jobs:**
 - When an action is invoked on an RDD, a job is created. Jobs are the main function submitted to Spark. Multiple jobs can run simultaneously within a Spark application if submitted from separate threads.

Execution Model

- **Stages:**

- The DAG scheduler creates multiple stages by splitting the RDD execution plan/DAG, typically created on shuffle boundaries. A stage is a sequence of transformations that can be executed in parallel on multiple nodes.

- **Schedulers:**

- The DAGScheduler operates at the stage level. The TaskScheduler operates at the task level. Resources for jobs are allocated in FIFO fashion by default.

- **Adaptive Query Execution:**

- Spark SQL adapts the execution plan at runtime, such as automatically setting the number of reducers and join algorithms.

Data Cleaning & Preprocessing

- Handle missing values, e.g., `df.na.fill()` method.

```
df_filled = df.na.fill({"age": mean_age})
```

- Apply feature scaling using `MinMaxScaler` or `StandardScaler`.

```
from pyspark.ml.feature import MinMaxScaler, StandardScaler,  
VectorAssembler
```

Exploratory Data Analysis (EDA)

- Provides us with insights into our data identifying patterns and make informed decisions.
- Perform tasks like summary statistics and Visualizations.

Exploratory Data Analysis (EDA)

- Provides a quick overview of the central tendencies and dispersion of the data.
- Calculates information like count, mean, standard deviation, minimum, and maximum for each column.
- Works only with integer columns.
- Use summary statistics with `df.describe()`

Exploratory Data Analysis (EDA)

- Utilize graphs and charts to understand data patterns visually.
- Visualizations through Pandas and Matplotlib, converting DataFrames with `toPandas()`
Perform correlation analysis with `stat.corr()`

SQL Functions in PySpark

- PySpark SQL functions allow you to transform data by applying operations to columns, such as arithmetic operations to create new columns or modify existing ones. They also enable string functions.
- To use these functions, you typically need to import them from `pyspark.sql.functions`
- Common functions include `col` to refer to a column, `concat` for string concatenation, and `lit` for literal values.

SQL Functions in PySpark

- PySpark SQL functions also enable you to manipulate data based on conditions and perform aggregations.
- Filtering allows you to retain only the rows that meet a certain condition. This is done using the **filter** or **where** methods of a DataFrame, often in conjunction with the **col** function to specify the column and the condition.
- Aggregation involves summarizing data. This can be done for the entire DataFrame or after grouping the data using the **groupBy** method. The **agg** method is used to apply aggregate functions. Functions like **avg**, **max**, **sum**, and **count** are commonly used aggregate functions, often imported from **pyspark.sql.functions**

GroupBy & Window Functions

- Use **groupBy** for aggregating data based on fields.
Apply window operations for advanced data analytics.
- They enable you to perform aggregations on subsets of your data and conduct advanced calculations across defined partitions or orderings within your DataFrame.
- Window operations allow you to perform calculations across a defined "window" or a set of rows that are related to the current row

```
df.groupBy("age").agg(avg("salary"), max("salary")).show()
```

User-Defined Functions (UDFs)

- Create UDFs for single and multi-column operations to extend functionality.
- This option allows you to extend PySpark's functionality by defining and applying custom data transformations and computations using Python.

```
def add_prefix(name):  
    return "Mr. " + name
```

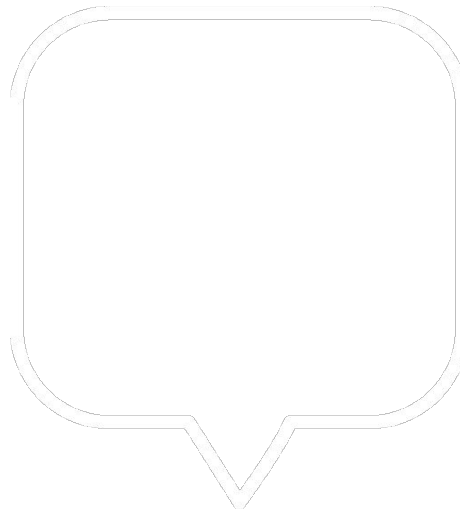
```
prefix_udf = udf(add_prefix, StringType())  
df.withColumn("prefixed_name",  
    prefix_udf(col("name"))).show()
```

Let's Code!



Hands-On Task

- Set up a local Spark environment using PySpark
- Load a static dataset (e.g., CSV, JSON)
- Apply batch ETL operations: filtering, transformation, and aggregation



Any Questions?



Quiz Time!

1. MLib is Spark's library specifically designed for:

- A. Graph processing
- B. Machine learning
- C. Structured data processing
- D. Real-time streaming

1. MLib is Spark's library specifically designed for:

- A. Graph processing
- B. Machine learning
- C. Structured data processing
- D. Real-time streaming

2. To begin using Spark MLlib in PySpark, your data is required to be in the form of a:
- A. Python list
 - B. RDD
 - C. Spark DataFrame
 - D. Pandas DataFrame

2. To begin using Spark MLlib in PySpark, your data is required to be in the form of a:

- A. Python list
- B. RDD
- C. **Spark DataFrame**
- D. Pandal DataFrame

3. For most high-level data processing tasks in PySpark, especially with structured data, which is the recommended entry point?

- A. SparkContext
- B. RDD API
- C. SparkConf
- D. SparkSession

3. For most high-level data processing tasks in PySpark, especially with structured data, which is the recommended entry point?

- A. SparkContext
- B. RDD API
- C. SparkConf
- D. **SparkSession**

4. A key advantage of Apache Spark compared to Hadoop MapReduce highlighted in the sources is its speed, often attributed to:

- A. In-memory data processing
- B. Disk-based computation
- C. Manual data shuffling
- D. Limited set of operations

4. A key advantage of Apache Spark compared to Hadoop MapReduce highlighted in the sources is its speed, often attributed to:

- A. **In-memory data processing**
- B. Disk-based computation
- C. Manual data shuffling
- D. Limited set of operations

5. What is the fundamental data structure in Apache Spark, described as an immutable, distributed collection of elements?

- A. DataFrame
- B. Dataset
- C. RDD (Resilient Distributed Dataset)
- D. Graph

5. What is the fundamental data structure in Apache Spark, described as an immutable, distributed collection of elements?

- A. DataFrame
- B. Dataset
- C. **RDD (Resilient Distributed Dataset)**
- D. Graph

Thanks